

Code Projects to Learn Claude More Effectively

Six hands-on projects tailored to a C / C++ / Python and microcontroller workflow.

1. Embedded Sensor Dashboard (C + Python)

Build firmware for an Arduino, STM32, or ESP32 that streams sensor data over serial or Wi-Fi, paired with a Python desktop dashboard. Use Claude to draft the C firmware, then iterate by pasting compiler errors and oscilloscope screenshots back into the chat. The lesson here is **tight feedback loops** — learning when to give Claude raw error output versus when to summarize, and how to keep context focused as the project grows across two languages.

2. Legacy C Codebase Refactor

Take an open-source C project (a small game engine, a parser, or an RTOS demo) and ask Claude to help you refactor it module by module — adding unit tests, modernizing to C11 / C++17, and documenting tribal knowledge. This teaches you how to **scope requests narrowly** ("refactor only this function, preserve this ABI"), how to use Claude for code review rather than generation, and how to verify its suggestions against a compiler.

3. Python CLI Tool with Test-Driven Development

Pick a personal utility you actually want — a log parser, a backup deduplicator, a GDB scripting helper — and build it strictly TDD-style with Claude. Write the test first (or have Claude write it), confirm it fails, then have Claude implement until it passes. You'll learn how Claude behaves differently when given **executable specifications** versus prose, and you'll get a feel for when to trust generated code versus stepping through it yourself.

4. Microcontroller Driver from a Datasheet

Attach a peripheral's PDF datasheet and have Claude help you write a register-level driver in C (e.g., an I²C IMU, an SPI display, a CAN transceiver). Datasheets are dense and easy to misread — you'll learn how to ask Claude to **cite specific page or register numbers**, how to catch hallucinated register addresses, and how to break a 200-page document into chunks Claude can reason about reliably.

5. A Claude-Powered Developer Tool

Use the Claude Agent SDK (Python) to build something for your own workflow: a code-review bot that runs on `git commit`, a "rubber duck" that watches your Visual Studio output window, or an MCP server that exposes your microcontroller's JTAG state to Claude. This flips the relationship — instead of using Claude as a chat partner, you're **programming against it**, which forces you to understand prompts, tools, context windows, and cost as engineering primitives.

6. Cross-Language Port (Bonus)

Port a non-trivial Python script to C++ (or vice versa) with Claude as your translator. Pick something with real data structures and edge cases — a JSON config parser, a small interpreter, an image filter. You'll learn how Claude handles **semantic equivalence across languages**, where it silently changes behavior (integer overflow, GC vs. RAI, exceptions vs. error codes), and how to write differential tests that prove the port is faithful.

Start with whichever sits closest to a problem you already have — projects that scratch a real itch teach you far more than blank-slate exercises, because you'll push back when Claude's suggestions don't fit your actual constraints.